

```

/**
 * =====
 * Slops Saloon Fantasy Football MVP (SSFFMVP)
 * Master Grade Agent Layer - v4
 * =====
 *
 * Integrates ssff-bot local logic into the v3 cloud architecture.
 * Four improvements over the previous agent layer:
 *
 * [A] LOCAL LOGIC BRIDGE - Emergency Override
 *     If Sleeper/Yahoo/ESPN APIs fail, the system ingests a JSON
 *     payload from the local ssff-bot scraper to keep the dashboard
 *     live. Automatic fallback chain: Cloud → Redis Cache → Local.
 *
 * [B] VORP MATH ENGINE - Performance Agent
 *     Before any AI reasoning runs, every player gets a hard
 *     Value Over Replacement Player (VORP) score computed from
 *     raw stats. The AI receives this as a "Math-Fact" block it
 *     cannot override. Prevents hallucinated hype.
 *
 * [C] POSITIONAL SCARCITY ANALYSIS - Waiver Agent
 *     Scans the full waiver pool across the league and calculates
 *     a Scarcity Multiplier per position. If only 2 viable QBs
 *     remain available, QB move confidence is auto-weighted up.
 *     Directly bridged from ssff-bot's positional logic.
 *
 * [D] MANAGER AGENT - Hardened Synthesis
 *     Receives Math-Facts (VORP), Scarcity context, and all 6
 *     agent signals. Produces the final JSON move with a
 *     confidence score that reflects both AI reasoning AND
 *     the mathematical inputs below it.
 *
 * Express routes added:
 *
 * POST /api/agents/run          - full agent pipeline for a user
 * POST /api/agents/local-ingest - receive ssff-bot JSON payload
 * GET  /api/agents/vorp/:leagueId:week - VORP table for a league
 * GET  /api/agents/scarcity/:leagueId - scarcity report
 *
 * Consumed by:
 *
 * ssffmvp_platform_integration.js (standings + roster)
 * ssffmvp_tuesday_cron.js         (outcome scoring)
 * =====
 */

```

```

"use strict";

const express          = require("express");
const { createClient } = require("@supabase/supabase-js");
const { Redis }        = require("@upstash/redis");
const Anthropic        = require("@anthropic/sdk");

const router  = express.Router();
const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_SERV
const redis    = new Redis({ url: process.env.REDIS_URL, token: process.env.REDIS
const ai       = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });
const MODEL    = "claude-3-5-sonnet-latest";

// ———— LOGGER ————
const ts = () => new Date().toISOString().replace("T", " ").slice(0, 19);
const log = {
  info: (...a) => console.log( `[${ts()}] `, ...a),
  ok:   (...a) => console.log( `[${ts()}] ✓`, ...a),
  warn: (...a) => console.warn( `[${ts()}] ⚠`, ...a),
  local: (...a) => console.log( `[${ts()}] `, ...a),
  math: (...a) => console.log( `[${ts()}] `, ...a),
  scarce: (...a) => console.log( `[${ts()}] `, ...a),
};

/*
=====
[A] LOCAL LOGIC BRIDGE
=====

The ssff-bot ran a local SQLite scraper that exported JSON
snapshots of rosters, standings, and waiver data. This module
accepts that payload as an emergency ingest and stores it in
Supabase as a "local_snapshot" row, stamped with its source.

Fallback chain for ANY data fetch in this module:
1. Cloud platform API (Sleeper/Yahoo/ESPN)
2. Redis cache (5 min TTL)
3. Local snapshot from ssff-bot ingest ← the bridge

The local payload schema mirrors ssff-bot's export format:
{
  leagueId: string,
  exportedAt: ISO string,
  source: "ssff-bot-local",
  rosters: [{ teamId, teamName, players: [{ id, name, pos, team, pts_last3: []
  waivers: [{ id, name, pos, team, pts_last3: [], owned_pct: number }],
  standings: [{ rank, teamName, wins, losses, pts }],
  matchups: [{ teamId, opponent, myPts, oppPts, week }]
}

```

```

/**
 * POST /api/agents/local-ingest
 * Accepts a JSON body matching the ssff-bot export format.
 * Validates schema, stores in Supabase local_snapshots table,
 * and purges the Redis cache for this league so next call
 * falls through to the fresh local data.
 */
router.post("/agents/local-ingest", async (req, res) => {
  const payload = req.body;

  // — Schema validation —
  const required = ["leagueId", "exportedAt", "rosters", "waivers", "standings"];
  const missing = required.filter(k => !payload[k]);
  if (missing.length) {
    return res.status(400).json({ error: `Missing fields: ${missing.join(", ")}` });
  }
  if (!Array.isArray(payload.rosters) || !Array.isArray(payload.waivers)) {
    return res.status(400).json({ error: "rosters and waivers must be arrays" });
  }

  const exportAge = Date.now() - new Date(payload.exportedAt).getTime();
  if (exportAge > 24 * 60 * 60 * 1000) {
    return res.status(400).json({ error: "Payload is older than 24 hours — re-exp" });
  }

  try {
    // Store snapshot
    const { error } = await supabase.from("local_snapshots").upsert({
      league_id: payload.leagueId,
      snapshot: payload,
      source: payload.source || "ssff-bot-local",
      exported_at: payload.exportedAt,
      ingested_at: new Date().toISOString(),
    }, { onConflict: "league_id" });

    if (error) throw new Error(`Supabase upsert failed: ${error.message}`);

    // Purge Redis so next standings/roster call picks up this data
    await Promise.allSettled([
      redis.del(`ssff:standings:${payload.leagueId}`),
      redis.del(`ssff:roster:*:${payload.leagueId}`),
      redis.del(`ssff:scarcity:${payload.leagueId}`),
    ]);

    log.local(`Ingested local snapshot for league ${payload.leagueId} — ${payload

```

```

    res.json({
      ok:      true,
      leagueId: payload.leagueId,
      rosters: payload.rosters.length,
      waivers: payload.waivers.length,
      note:    "Redis cache purged – next API call will use this snapshot as fal
    });
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});

/**
 * Retrieves roster/waiver data using the full fallback chain.
 * Returns { data, source: "cloud" | "redis" | "local" }
 */
async function fetchWithLocalFallback(leagueId, dataType, cloudFetcher) {
  const cacheKey = `ssff:${dataType}:${leagueId}`;

  // 1. Try Redis cache
  try {
    const cached = await redis.get(cacheKey);
    if (cached) {
      const parsed = typeof cached === "string" ? JSON.parse(cached) : cached;
      log.info(`[Fallback:${dataType}] Redis HIT`);
      return { data: parsed, source: "redis" };
    }
  } catch (e) { log.warn(`Redis get failed (${cacheKey}): ${e.message}`); }

  // 2. Try cloud API
  try {
    const data = await cloudFetcher();
    await redis.set(cacheKey, JSON.stringify(data), { ex: 300 }).catch(() => null);
    log.info(`[Fallback:${dataType}] Cloud OK`);
    return { data, source: "cloud" };
  } catch (cloudErr) {
    log.warn(`[Fallback:${dataType}] Cloud failed: ${cloudErr.message} – trying l
  }

  // 3. Emergency: Local ssff-bot snapshot
  const { data: row } = await supabase
    .from("local_snapshots")
    .select("snapshot, ingested_at, source")
    .eq("league_id", leagueId)
    .single();

  if (!row?.snapshot) {

```

```

    throw new Error(`All data sources failed for ${dataType} (league ${leagueId})
  }

const ageHrs = ((Date.now() - new Date(row.ingested_at).getTime()) / 3600000).toFixed(1)
log.local(`[Fallback:${dataType}] Using local snapshot (age: ${ageHrs}h, source

const localData = dataType === "waivers"    ? row.snapshot.waivers
                  : dataType === "rosters"   ? row.snapshot.rosters
                  : dataType === "standings" ? row.snapshot.standings
                  : null;

if (!localData) throw new Error(`dataType "${dataType}" not found in local snap

return { data: localData, source: `local:${row.source}`, snapshotAge: `${ageHrs}h` }
}

/* =====
[B] VORP MATH ENGINE
=====

Value Over Replacement Player (VORP) is the industry-standard
metric for judging whether a player is worth starting/adding.

Formula:
  VORP = playerAvgPts - replacementBaseline[position][scoring]

Replacement baselines (PPR) – the expected score from the
best available free agent at each position:
  QB: 18.0    (streaming QB is viable, so baseline is higher)
  RB: 8.5     (RB scarcity drives the value gap)
  WR: 9.0
  TE: 7.0     (most TEs are replacement-level)
  K: 7.5
  DEF: 7.0

These baselines are calibrated for PPR. Half PPR and Standard
shift them by the receptions adjustment below.

The VORP block is injected into the Manager Agent prompt as a
"MATH-FACT" section the AI is explicitly instructed not to
contradict without strong positional reasoning.
===== */

// Replacement baselines by scoring format
const REPLACEMENT_BASELINES = {
  PPR: {
    QB: 18.0, RB: 8.5, WR: 9.0, TE: 7.0, K: 7.5, DEF: 7.0,
  },

```

```

    "Half PPR": {
      QB: 17.5, RB: 8.0, WR: 8.5, TE: 6.5, K: 7.5, DEF: 7.0,
    },
    Standard: {
      QB: 17.0, RB: 7.5, WR: 8.0, TE: 6.0, K: 7.5, DEF: 7.0,
    },
    "2QB / Superflex": {
      QB: 22.0, RB: 8.5, WR: 9.0, TE: 7.0, K: 7.5, DEF: 7.0,
    },
    "TE Premium": {
      QB: 18.0, RB: 8.5, WR: 9.0, TE: 11.0, K: 7.5, DEF: 7.0,
    },
  };

// Trend multipliers - reward upward trajectories
// Players on a hot streak get a momentum bonus on their baseline
function calcTrendMultiplier(ptsLast3 = []) {
  if (ptsLast3.length < 2) return 1.0;
  const diffs = ptsLast3.slice(1).map((v, i) => v - ptsLast3[i]);
  const avgDiff = diffs.reduce((a, d) => a + d, 0) / diffs.length;
  // +/- 1% per point of avg weekly improvement
  return Math.max(0.85, Math.min(1.20, 1 + avgDiff * 0.01));
}

/**
 * Calculates VORP for a single player.
 * @param {object} player - { name, pos, pts_last3: number[], pts_season_avg?: n
 * @param {string} scoring - "PPR" | "Half PPR" | "Standard" | etc.
 * @returns {object} - { vorp, avgPts, baseline, trend, grade }
 */
function calcVORP(player, scoring = "PPR") {
  const baselines = REPLACEMENT_BASELINES[scoring] || REPLACEMENT_BASELINES.PPR
  const baseline = baselines[player.pos] ?? 8.0;
  const pts = player.pts_last3 || [];
  const avgPts = pts.length
    ? pts.reduce((a, v) => a + v, 0) / pts.length
    : (player.pts_season_avg || 0);
  const trend = calcTrendMultiplier(pts);
  const trendedAvg = avgPts * trend;
  const vorp = parseFloat((trendedAvg - baseline).toFixed(2));

  // Letter grade: makes the Math-Fact block human-readable for the AI
  const grade = vorp >= 8 ? "A+"
    : vorp >= 5 ? "A"
    : vorp >= 3 ? "B+"
    : vorp >= 1 ? "B"
    : vorp >= -1 ? "C"

```

```

        : vorp >= -3 ? "D"
        : "F";

    return { vorp, avgPts: parseFloat(avgPts.toFixed(2)), baseline, trend: parseFlo
}

/**
 * Builds a VORP table for a full roster (or waiver list).
 * Returns an array sorted by VORP descending.
 */
function buildVORPTable(players, scoring) {
    return players
        .map(p => ({ ...p, ...calcVORP(p, scoring) }))
        .sort((a, b) => b.vorp - a.vorp);
}

/**
 * Formats the VORP table as a compact Math-Fact string
 * to inject into the Manager Agent prompt.
 */
function formatVORPBlock(vorpTable, label = "ROSTER") {
    const header = `MATH-FACTS: ${label} VORP TABLE (${new Date().toISOString().sli
    const sep      = "-".repeat(62);
    const rows     = vorpTable.slice(0, 15).map(p =>
        ` ${p.grade.padEnd(3)} ${String(p.vorp >= 0 ? "+" + p.vorp : p.vorp).padStar
    ).join("\n");
    return `${header}\n${sep}\n${rows}\n${sep}\nNOTE: These are calculated facts. D
}

// GET /api/agents/vorp/:leagueId/:week
router.get("/api/agents/vorp/:leagueId/:week", async (req, res) => {
    const { leagueId, week } = req.params;
    const scoring = req.query.scoring || "PPR";
    const cacheKey = `ssff:vorp:${leagueId}:${week}:${scoring}`;

    try {
        const cached = await redis.get(cacheKey).catch(() => null);
        if (cached) return res.json(typeof cached === "string" ? JSON.parse(cached) :

        // Fetch waiver pool
        const { data: waivers } = await fetchWithLocalFallback(leagueId, "waivers", a
            const r = await fetch(`https://api.sleeper.app/v1/league/${leagueId}/player
            if (!r.ok) throw new Error(`Sleeper waivers ${r.status}`);
            return r.json();
        });

        const vorpTable = buildVORPTable(waivers, scoring);

```

```

    const result    = { leagueId, week, scoring, vorpTable, generatedAt: new Date
    await redis.set(cacheKey, JSON.stringify(result), { ex: 600 }).catch(() => nu
    log.math(`VORP table built: ${vorpTable.length} players, league ${leagueId}`)
    res.json(result);
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});

```

```

/* =====
[C] POSITIONAL SCARCITY ANALYSIS
=====

```

Bridged from ssff-bot's positional logic. The original bot counted viable players per position on the waiver wire and used that count to flag "scarce" positions.

This implementation formalizes that into a Scarcity Score and Confidence Multiplier:

```

Scarcity Score = viableCount / leagueSize
  < 0.5  → Critical scarcity  (multiplier: +20 confidence pts)
  < 1.0  → High scarcity      (multiplier: +12 confidence pts)
  < 2.0  → Moderate scarcity  (multiplier: +5 confidence pts)
  ≥ 2.0  → Adequate depth     (multiplier: 0)

```

"Viable" definition: VORP > 0 (positive value over replacement)
 This is stricter than just "available" – a QB with VORP -4 doesn't count as a viable add regardless of availability.

The scarcity report is injected into the Manager Agent prompt as a second Math-Fact block alongside VORP.

```

===== */

```

```

const SCARCITY_THRESHOLDS = [
  { label: "CRITICAL", maxRatio: 0.5, confidenceBonus: 20, emoji: "🔴" },
  { label: "HIGH",      maxRatio: 1.0, confidenceBonus: 12, emoji: "⚠️" },
  { label: "MODERATE",  maxRatio: 2.0, confidenceBonus: 5,  emoji: "🟡" },
  { label: "ADEQUATE",  maxRatio: Infinity, confidenceBonus: 0, emoji: "✅" },
];

```

```

const POSITIONS_TO_TRACK = ["QB", "RB", "WR", "TE"];

```

```

/**
 * Builds a full scarcity report for every tracked position.
 * @param {Array}  waivers      – waiver pool players with VORP already calculated
 * @param {number} leagueSize – number of teams in the league
 * @returns {object} scarcityReport

```

```

*/
function buildScarcityReport(waivers, leagueSize = 10, scoring = "PPR") {
  const vorpPool = buildVORPTable(waivers, scoring);

  const report = {};
  for (const pos of POSITIONS_TO_TRACK) {
    const viable = vorpPool.filter(p => p.pos === pos && p.vorp > 0);
    const total = vorpPool.filter(p => p.pos === pos);
    const ratio = viable.length / leagueSize;
    const tier = SCARCITY_THRESHOLDS.find(t => ratio < t.maxRatio) || SCARCITY_

    report[pos] = {
      position:      pos,
      viableCount:    viable.length,
      totalAvailable: total.length,
      leagueSize,
      ratio:          parseFloat(ratio.toFixed(2)),
      scarcityLabel:  tier.label,
      confidenceBonus: tier.confidenceBonus,
      emoji:          tier.emoji,
      topAvailable:   viable.slice(0, 3).map(p => ({ name: p.name, vorp: p.vorp,
    });
  }

  return report;
}

/**
 * Formats the scarcity report as a Math-Fact string for the AI prompt.
 */
function formatScarcityBlock(report) {
  const lines = Object.values(report).map(r =>
    ` ${r.emoji} ${r.position.padEnd(3)}  ${r.scarcityLabel.padEnd(10)}  viable:
  );
  return [
    "MATH-FACTS: POSITIONAL SCARCITY ANALYSIS",
    "-".repeat(62),
    ...lines,
    "-".repeat(62),
    "NOTE: For any move involving a CRITICAL or HIGH scarcity position,",
    "      add the confidenceBonus to your base confidence score.",
    "      Example: QB move with base 70 confidence + 20 CRITICAL bonus = 90.",
  ].join("\n");
}

// GET /api/agents/scarcity/:leagueId
router.get("/agents/scarcity/:leagueId", async (req, res) => {

```

```

const { leagueId } = req.params;
const scoring      = req.query.scoring      || "PPR";
const leagueSize   = parseInt(req.query.leagueSize || "10");
const cacheKey     = `ssff:scarcity:${leagueId}:${scoring}`;

try {
  const cached = await redis.get(cacheKey).catch(() => null);
  if (cached) return res.json(typeof cached === "string" ? JSON.parse(cached) :

  const { data: waivers, source } = await fetchWithLocalFallback(leagueId, "wai
    const r = await fetch(`https://api.sleeper.app/v1/league/${leagueId}/player
    if (!r.ok) throw new Error(`Sleeper waivers ${r.status}`);
    return r.json();
  });

  const report = buildScarcityReport(waivers, leagueSize, scoring);
  const result = { leagueId, scoring, leagueSize, source, report, generatedAt:
  await redis.set(cacheKey, JSON.stringify(result), { ex: 300 }).catch(() => nu
  log.scarce(`Scarcity report: ${Object.keys(report).map(p => `${p}:${report[p]
  res.json(result);
} catch (e) {
  res.status(500).json({ error: e.message });
}
});

```

```

/* =====
[D] MANAGER AGENT – Hardened Synthesis
=====

```

Receives:

- 6 agent signal strings (Weather, Travel, GameTime, Roster, Perf, Matchup)
- VORP Math-Fact block [B]
- Scarcity Math-Fact block [C]
- Historical calibration from self-improving loop
- User's scoring format + team context

Returns JSON move with confidence score that reflects:

- AI reasoning over all signals
- VORP ranking of the target player
- Scarcity confidence bonus for the target position
- Historical win-rate calibration

The system prompt explicitly chains the math facts FIRST,
 then allows AI reasoning, then applies the scarcity bonus.
 This ordering prevents the AI from ignoring cold numbers.

```

===== */

```

```

/**

```

```

* Fetches the self-improving calibration context from Supabase.
* Falls back to empty string if not yet populated.
*/
async function getCalibrationContext() {
  try {
    const { data } = await supabase
      .from("system_context")
      .select("value")
      .eq("key", "manager_agent_calibration")
      .single();
    return data?.value?.prompt || "";
  } catch { return ""; }
}

/**
* Main hardened agent pipeline.
* Called by POST /api/agents/run
*/
async function runAgentPipeline(context) {
  const {
    leagueId, userId, scoring, leagueSize,
    week, record,
    signals, // { weather, travel, gametime, roster, perf, matchup }
  } = context;

  log.info(`Agent pipeline: league ${leagueId} week ${week} scoring ${scoring}`);

  // — Parallel data fetch: VORP + Scarcity + Calibration —
  const [vorpResult, scarcityResult, calibration] = await Promise.allSettled([
    // [B] VORP
    (async () => {
      const { data: waivers, source } = await fetchWithLocalFallback(leagueId, "w
      const r = await fetch(`https://api.sleeper.app/v1/league/${leagueId}/play
      if (!r.ok) throw new Error(`Sleeper ${r.status}`);
      return r.json();
    })();
    log.math(`VORP source: ${source}`);
    return { table: buildVORPTable(waivers, scoring), source };
  ]),

    // [C] Scarcity
    (async () => {
      const { data: waivers, source } = await fetchWithLocalFallback(leagueId, "w
      const r = await fetch(`https://api.sleeper.app/v1/league/${leagueId}/play
      if (!r.ok) throw new Error(`Sleeper ${r.status}`);
      return r.json();
    })();

```

```

    return { report: buildScarcityReport(waivers, leagueSize, scoring), source
    })(),

    // Historical calibration
    getCalibrationContext(),
  ]);

const vorpTable      = vorpResult.status      === "fulfilled" ? vorpResult.value.
const scarcityRpt    = scarcityResult.status === "fulfilled" ? scarcityResult.va
const calibText      = calibration.status     === "fulfilled" ? calibration.value
const dataSource     = vorpResult.status      === "fulfilled" ? vorpResult.value.

// Format math blocks
const vorpBlock      = vorpTable.length      ? formatVORPBlock(vorpTable, "WAIVER W
const scarcityBlock = Object.keys(scarcityRpt).length ? formatScarcityBlock(sca

// [D] Build hardened system prompt
const systemPrompt = `You are the Slops Saloon Fantasy Football MVP Manager Age

CRITICAL INSTRUCTION – READ BEFORE REASONING:
You will receive two MATH-FACT blocks below. These are calculated from real stati
You MUST base your confidence score on the math before applying narrative reasoni
Do NOT contradict a positive VORP grade with a low confidence score without expli
Do NOT ignore a CRITICAL scarcity bonus – add it to your base confidence.

${vorpBlock}

${scarcityBlock}

${calibText ? `HISTORICAL CALIBRATION:\n${calibText}` : ""}

OUTPUT FORMAT – return ONLY valid JSON, no markdown:
{
  "moveType":      string, // "Waiver Pickup" | "Start/Sit" | "Trade" | "Stack"
  "headline":      string, // ≤ 8 words, action-oriented
  "targetPlayer":  string, // primary player name
  "targetPosition": string, // QB | RB | WR | TE | K | DEF
  "confidence":    number, // 0-100, MUST reflect VORP grade + scarcity bonus
  "vorpGrade":     string, // copy the grade from MATH-FACTS for the target pla
  "scarcityBonus": number, // bonus points added from scarcity (0 if adequate)
  "reasoning":     string, // 2-3 sentences citing agent signals AND math facts
  "shortVerdict":  string, // 1 sharp sentence
  "dataSource":    string  // "${dataSource}"
}

SCORING FORMAT: ${scoring} – this changes which positions have highest VORP impac
TEAM RECORD: ${record}`;

```

```

const userPrompt = `AGENT INTELLIGENCE REPORT - Week ${week}:
WEATHER:    ${signals.weather    || "N/A"}
TRAVEL:     ${signals.travel     || "N/A"}
GAME TIME:  ${signals.gametime   || "N/A"}
ROSTER:     ${signals.roster     || "N/A"}
PERFORMANCE: ${signals.perf      || "N/A"}
MATCHUP:    ${signals.matchup    || "N/A"}

```

Using the MATH-FACT blocks in your system prompt alongside these agent signals, generate the single best move for this GM this week.`;

```

// — AI Call —————
let move;
try {
  const res = await ai.messages.create({
    model:      MODEL,
    max_tokens: 1000,
    system:     systemPrompt,
    messages:   [{ role: "user", content: userPrompt }],
  });
  const raw    = res.content.map(b => b.text || "").join("").replace(/```json|```/g, "");
  move        = JSON.parse(raw);
  log.ok(`Manager Agent: ${move.moveType} - ${move.headline} (confidence: ${move.confidence})`);
} catch (e) {
  log.warn(`Manager Agent AI failed: ${e.message} - using VORP-based fallback`);
  // [B][C] Fallback: use pure math to generate a move without AI
  move = buildMathFallbackMove(vorpTable, scarcityRpt, scoring, dataSource);
}

return { move, vorpTable: vorpTable.slice(0, 10), scarcityReport: scarcityRpt,
}

/**
 * [B][C] Pure-math fallback move - used when AI call fails.
 * Picks the highest-VORP waiver add at the scarcest position.
 * No AI required - cold numbers only.
 */
function buildMathFallbackMove(vorpTable, scarcityRpt, scoring, dataSource) {
  // Find position with highest scarcity bonus
  const positions = Object.values(scarcityRpt).sort((a, b) => b.confidenceBonus);
  const scarcePos = positions[0];
  const bestPlayer = vorpTable.find(p => p.pos === (scarcePos?.position || "RB")
    || vorpTable.find(p => p.vorp > 0)
    || vorpTable[0]);

  if (!bestPlayer) {

```

```

    return {
      moveType: "Lineup Tweak", headline: "Review Your Starting Lineup",
      targetPlayer: "N/A", targetPosition: "N/A",
      confidence: 50, vorpGrade: "N/A", scarcityBonus: 0,
      reasoning: "Insufficient data for a math-backed recommendation this week.",
      shortVerdict: "Check back once waiver data refreshes.",
      dataSource,
    };
  }

const scarcityBonus = scarcePos?.confidenceBonus || 0;
const baseConf      = Math.min(95, Math.max(40, 50 + bestPlayer.vorp * 3));
const confidence     = Math.min(99, Math.round(baseConf + scarcityBonus));

return {
  moveType:      "Waiver Pickup",
  headline:      `Add ${bestPlayer.name.split(" ").pop()}, Drop Bench Piece`,
  targetPlayer:  bestPlayer.name,
  targetPosition: bestPlayer.pos,
  confidence,
  vorpGrade:     bestPlayer.grade,
  scarcityBonus,
  reasoning:     `VORP grade ${bestPlayer.grade} (+${bestPlayer.vorp}) at ${be
  shortVerdict:  `Best available by VORP at the scarcest position - add immedi
  dataSource,
};
}

/* =====
  MAIN PIPELINE ROUTE
  ===== */
router.post("/agents/run", async (req, res) => {
  const {
    leagueId, userId, scoring = "PPR",
    leagueSize = 10, week, record = "N/A",
    signals = {},
  } = req.body;

  if (!leagueId || !week) {
    return res.status(400).json({ error: "leagueId and week are required" });
  }

  try {
    const result = await runAgentPipeline({ leagueId, userId, scoring, leagueSize
    res.json({ ok: true, ...result });
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
}

```

```

    }
  });

module.exports = {
  router,
  // Exported for use in tuesday_cron.js and other modules
  calcVORP,
  buildVORPTable,
  buildScarcityReport,
  formatVORPBlock,
  formatScarcityBlock,
  fetchWithLocalFallback,
  runAgentPipeline,
};

/*
=====
SUPABASE SCHEMA ADDITIONS (run once)
=====

create table if not exists local_snapshots (
  league_id    text primary key,
  snapshot     jsonb not null,
  source       text default 'ssff-bot-local',
  exported_at  timestampz,
  ingested_at  timestampz default now()
);

=====
MOUNT IN EXPRESS APP
=====

const { router: agentRoutes } = require("./ssffmvp_agents");
app.use("/api", agentRoutes);

=====
ssff-bot LOCAL EXPORT FORMAT (for POST /api/agents/local-ingest)
=====

{
  "leagueId":    "1048209827491627008",
  "exportedAt":  "2025-11-12T08:30:00.000Z",
  "source":      "ssff-bot-local",
  "rosters": [
    {
      "teamId":   "1",
      "teamName": "Gridiron Gods",

```

```

    "players": [
      { "id":"4046", "name":"Derrick Henry", "pos":"RB", "team":"BAL",
        "pts_last3": [24.1, 18.6, 31.2], "pts_season_avg": 22.4 }
    ]
  },
  "waivers": [
    { "id":"9509", "name":"Jaylen Warren", "pos":"RB", "team":"PIT",
      "pts_last3": [15.2, 16.8, 22.1], "pts_season_avg": 12.1, "owned_pct": 38
    },
    { "id":"11592", "name":"Bo Nix", "pos":"QB", "team":"DEN",
      "pts_last3": [19.4, 22.6, 24.1], "pts_season_avg": 18.9, "owned_pct": 61
    }
  ],
  "standings": [
    { "rank":1, "teamName":"Gridiron Gods", "wins":11, "losses":3, "pts":"1842.
  ],
  "matchups": [
    { "teamId":"2", "opponent":"Gridiron Gods", "myPts":142.6, "oppPts":138.2,
  ]
}

```

PACKAGES

```

npm i @supabase/supabase-js @anthropic-ai/sdk @upstash/redis express
*/

```